

PDE-methods for Image Processing

Jake Lee, Luke Nowlis, Xiang Wan

USRE 2025

Contents

1	Preliminaries	2
1.1	Calculus/Analysis	2
1.2	Numerics	4
2	PDE	9
2.1	Heat equation (isotropic diffusion)	9
2.2	Anisotropic Diffusion	14
3	Variational Methods	16
3.1	Euler-Lagrange equation	17
3.2	Application 1: Denoising (Restoration)	18
3.3	Choice of function for Non linear Diffusion	19

1. Preliminaries

1.1. Calculus/Analysis

The concept of ‘magnitude/length’ is very important in applications, as it has been demonstrated for vectors in Calculus, especially for optimization problems. To quantitatively study images, we need to extend such concepts. The common tools used in math are norms.

Definition 1. Given a vector space $\mathcal{V}$, a *norm* is a non-negative real-valued function $\|\cdot\| : \mathcal{V} \rightarrow [0, \infty)$ such that for $u, v \in \mathcal{V}$ and $\alpha \in \mathbb{R}$,

- (Positive Definiteness) $\|u\| \geq 0$, and $\|u\| = 0$ if and only if $u = 0$.
- (Absolute Homogeneity) $\|\alpha u\| = |\alpha| \|u\|$.
- (Triangle Inequality) $\|u + v\| \leq \|u\| + \|v\|$.

Further reading: [https://en.wikipedia.org/wiki/Norm_\(mathematics\)](https://en.wikipedia.org/wiki/Norm_(mathematics))

Definition 2. A vector space with a norm defined is called a *normed vector space*. The *unit sphere* of a normed vector space $\mathcal{V}$ is the set of unit vectors: $\mathcal{S} = \{u \in \mathcal{V} : \|u\| = 1\}$. The *unit ball* of a normed vector space $\mathcal{V}$ is the set of vectors: $\mathcal{B} = \{u \in \mathcal{V} : \|u\| \leq 1\}$.

Problem 1.1. Recall from Calculus that for the Euclidean space $\mathcal{V} = \mathbb{R}^n$ and a vector $\mathbf{u} = [u_1, \dots, u_n] \in \mathbb{R}^n$, we can define three norms:

- The 2-norm (Euclidean norm):

$$\|\mathbf{u}\|_2 = |\mathbf{u}| = \left(\sum_{i=1}^n u_i^2 \right)^{\frac{1}{2}}. \quad (1.1)$$

- The 1-norm:

$$\|\mathbf{u}\|_1 = \sum_{i=1}^n |u_i|. \quad (1.2)$$

- The sup-norm (or ∞ -norm):

$$\|\mathbf{u}\|_\infty = \max_{1 \leq i \leq n} |u_i|. \quad (1.3)$$

- Verify the three ‘norms’ above are indeed norms according to Definition 1.
- For $\mathbb{R}^2$, draw the graphs of unit balls associated with the three norms above.
- How about $\mathbb{R}^3$? How about $\mathbb{R}^n$?

Coding Problem 1.2. You should get used to the *numpy* module in python, which specializes in vector operations. Many other advanced packages in python depend on *numpy*. In particular, it’s very common to use the *numpy-array* data structure to store and operate vectors.

Find the correct syntax to define the vector $\mathbf{v} = [1, 3, 5, \dots, 2345]$ (not typed in manually, obviously). Print out its dimension, and compute the three aforementioned norms of $\mathbf{v}$ via built-in functions in *numpy* (Google is your friend).

Definition 3. Given two functions $f(x)$ and $g(x)$, their *convolution* is defined by

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x - \tau)g(\tau) d\tau. \quad (1.4)$$

Remark 1.1. The convolution is defined via an improper integral and therefore may be divergent. We say it is *well defined* if the integral is convergent.

Problem 1.3. Argue that $f * g = g * f$, that is,

$$\int_{-\infty}^{\infty} f(\tau)g(x - \tau) d\tau$$

SOLUTION: The two equations represent the convolution of $(f * g)$ and $(g * f)$.

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x - \tau)g(\tau)d\tau.$$

$$(g * f)(x) = \int_{-\infty}^{\infty} f(\tau)g(x - \tau)(d\tau).$$

I argue that these two have the commutativity property.

First, I define the convolution from f to g .

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x - \tau)g(\tau)d\tau.$$

Using U-Sub,

We make

$$u = x - \tau.$$

Rewrite this as

$$\tau = x - u$$

.

Deriving both sides with respect to u , I get

$$\begin{aligned} \frac{d\tau}{du} &= \frac{d}{du}(x - u) \\ d\tau &= -du \end{aligned}$$

Since I changed the variable that is being integrated, I redefined the boundaries of the integration with respect to u by

When

$$\tau = -\infty, u = x - (-\infty) = \infty$$

When

$$\tau = \infty, u = x - \infty = -\infty$$

Becoming

$$(f * g)(x) = \int_{\infty}^{-\infty} f(x - u)g(u)(-du).$$

Since -1 is a constant, I can take it out of the integration

$$(f * g)(x) = - \int_{-\infty}^{\infty} f(x-u)g(u)(du)$$

The definition of definite integral states that $-\int_a^b f(x) = \int_b^a f(x)$

Therefore,

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x-u)g(u)(du)$$

Replace

$$x-u$$

for u

$$(f * g)(x) = \int_{-\infty}^{\infty} f(x-(x-\tau))g(x-\tau)(d\tau).$$

Shows that

$$(f * g)(x) = \int_{-\infty}^{\infty} f(\tau)g(x-\tau)(d\tau) = (g * f)(x) = \int_{-\infty}^{\infty} f(\tau)g(x-\tau)(d\tau).$$

Definition 4. Given two functions $f(x, y)$ and $g(x, y)$, their convolution is defined as

$$(f * g)(x, y) = \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} f(u, v) \cdot g(x-u, y-v) du dv \quad (1.5)$$

whenever the integral is convergent. The convolution satisfies (no verification needed for now)

- **Commutativity:**

$$f * g = g * f$$

- **Associativity:**

$$(f * g) * h = f * (g * h)$$

- **Linearity:**

$$f * (ag_1 + bg_2) = a(f * g_1) + b(f * g_2)$$

A Gaussian kernel:

$$g(x, y) = \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

1.2. Numerics

Definition 5. For a convergent sequence $\{x_n\}$ (with $\lim_{n \rightarrow \infty} x_n = a$, or in short $x_n \rightarrow a$),

- the *absolute error* at the n -th step is

$$|x_n - a|$$

- the *consecutive/successive error* at the n -th step is

$$|x_n - x_{n-1}|$$

- the *relative error* at the n -th step is

$$\left| \frac{x_n - x_{n-1}}{x_n} \right| \quad \text{or} \quad \left| \frac{x_n - x_{n-1}}{x_{n-1}} \right|$$

Further reading: https://en.wikipedia.org/wiki/Approximation_error

Problem 1.4. (a) Argue that when $x_n \rightarrow a$, both the absolute error and the consecutive error go to 0.

(b) In addition, if $a \neq 0$, argue that the relative error also goes to 0.

(c) Finally, found a counter-example of a convergent sequence whose relative error does not go to 0.

Definition 6. For a function $f(x) : \mathbb{R} \rightarrow \mathbb{R}$ and some initial guess x_0 for a root r of the function (such that $f(r) = 0$), the *Newton's method* is the iteration scheme defined by:

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)} \quad (1.6)$$

whenever $f'(x_n) \neq 0$. We say the method is *convergent* if $x_n \rightarrow r$.

Algorithm 1 Newton's method

Require: $max_iteration > 0$

```

1:  $r \leftarrow x_0$ 
2:  $i = 0$ 
3: while  $i \leq max\_iteration$  and stopping criterion is False do
4:    $r \leftarrow r - \frac{f(r)}{f'(r)}$ 
5:    $i \leftarrow i + 1$ 
6: end while
7: return  $r$ 
```

Read more here: https://en.wikipedia.org/wiki/Newton%27s_method.

Coding Problem 1.5. Implement Newton's method to $f(x) = x^3 + x - 1 = 0$. Try both the successive error and the relative error for the stopping criterion.

Extra question: can we use the absolute error?

Coding Problem 1.6. Implement Newton's method to $f(x) = 4x^4 - 6x^2 + 3x - \frac{11}{4}$.

Coding Problem 1.7. We can now use the Newton's method to find the minima of $g(x)$. From Calculus, it means we are looking for critical points where $g'(x) = 0$.

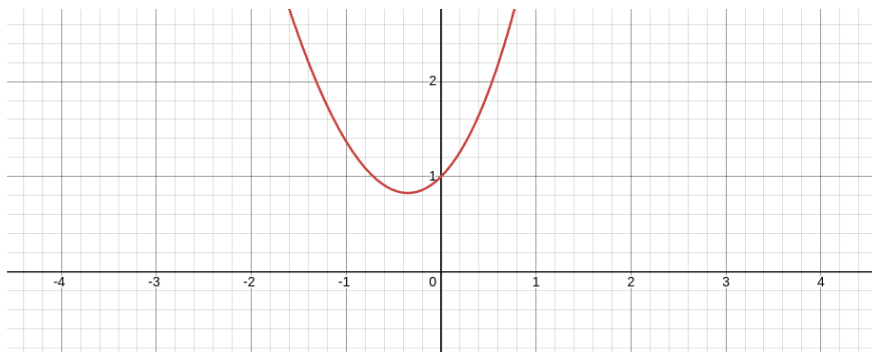
Answer the following questions.

- (a) Write down the Newton's method formula for critical points of $g(x)$.

(b) Implement the formula you came up with in part (a). Try it with

$$g_1(x) = e^x + x^2,$$

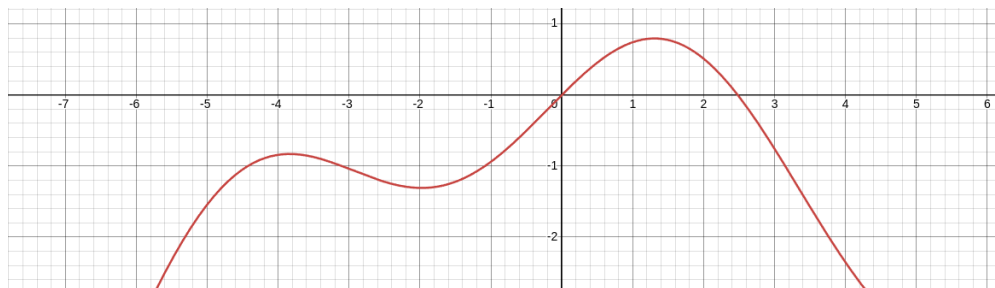
whose graph is:



(c) Try it with

$$g_2(x) = \sin(x) - 0.1x^2,$$

whose graph is:



Report the x -values of all local maxima and minima in the picture above.

Your code needs to automatically tell which one(s) are local max and which are local min.

Definition 7. Consider now a multivariate function¹: $f(\boldsymbol{\theta}) : \mathbb{R}^n \rightarrow \mathbb{R}$. The input $\boldsymbol{\theta}$ is often referred to as the *parameter vector*.

The *Gradient Descent* is an iterative optimization algorithm used to minimize the function $f(\boldsymbol{\theta})$ by moving in the direction of the steepest descent, as defined by the negative of the gradient². The update rule at each iteration t is:

$$\boldsymbol{\theta}_{n+1} = \boldsymbol{\theta}_n - \eta \cdot \nabla f(\boldsymbol{\theta}_n) \quad (1.7)$$

where:

- $\boldsymbol{\theta}_n$ is the parameter vector at iteration n , with $\boldsymbol{\theta}_0$ being an initial guess.
- η is the learning rate (step size),

¹also called a *function of multiple variables*, read more here: https://en.wikipedia.org/wiki/Function_of_several_real_variables

²this is from MATH 162

- $\nabla f(\boldsymbol{\theta}_n)$ is the gradient of the objective function with respect to $\boldsymbol{\theta}$ at $\boldsymbol{\theta}_n$.
- Stopping criterion is when $\nabla f(\boldsymbol{\theta}_n) \approx 0$ or, numerically,

$$|\nabla f(\boldsymbol{\theta}_n)| \leq \text{TOL}, \quad \text{TOL is a pre-selected tolerance level at, for instance, } 10^{-6}.$$

The algorithm converges when the gradient approaches zero or after a specified number of iterations. Read more here: https://en.wikipedia.org/wiki/Gradient_descent. An exemplary implementation can be found [here](#).

Algorithm 2 Gradient Descent

Require: max_iterations > 0, $\eta > 0$

```

1:  $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta}_0$ 
2:  $i \leftarrow 0$ 
3: while  $i \leq \text{max\_iterations}$  and stopping criterion is False do
4:    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla f(\boldsymbol{\theta})$ 
5:    $i \leftarrow i + 1$ 
6: end while
7: return  $\boldsymbol{\theta}$ 

```

Coding Problem 1.8. Redo Problem 1.7 via the Gradient Descent.

Extra question: does the choice of η matter?

Coding Problem 1.9. Use gradient descent to minimize the following function:

$$f(\boldsymbol{\theta}) = \theta_1^2 + 4\theta_2^2 + 2\theta_1\theta_2 + 3\theta_1 - \theta_2 + 5 \quad (1.8)$$

where $\boldsymbol{\theta} = \begin{bmatrix} \theta_1 \\ \theta_2 \end{bmatrix} = [\theta_1, \theta_2]^T \in \mathbb{R}^2$. Compare your result with the analytic answer by techniques from MATH 162.

Coding Problem 1.10. Use gradient descent to minimize the following function in $\mathbb{R}^4$:

$$f(\boldsymbol{\theta}) = \theta_1^2 + 3\theta_2^2 + 2\theta_3^2 + \theta_4^2 + \theta_1\theta_2 - 2\theta_2\theta_3 + \theta_3\theta_4 - 4\theta_1 + \theta_2 + 2\theta_3 - \theta_4 + 10 \quad (1.9)$$

where $\boldsymbol{\theta} = [\theta_1, \theta_2, \theta_3, \theta_4]^T$.

Try

- (a) $\eta = 1$
- (b) $\eta = 0.5$
- (c) $\eta = 0.1$
- (d) $\eta = 0.001$

You may try a few more. What did you observe?

Definition 8. A (numerical) quadrature is an approximation of a definite integral $\int_a^b f(x) dx$. A natural way to define a quadrature is to first introduce a partition of the integration interval $[a, b]$. The easiest way is to use a uniform grid³ of N sub-intervals. We then define

$$a = x_0 < x_1 < \cdots < x_n = b, \quad h = \frac{b-a}{n}, \quad x_{i+1} = x_i + h \quad (1.10)$$


Then some quadratures from MATH 161 are:

- *left endpoints approximation:*

$$\int_a^b f(x) dx \approx h \cdot \underbrace{\sum_{i=1}^n f(x_{i-1})}_{\text{sum of left heights}}$$

- *right endpoints approximation:*

$$\int_a^b f(x) dx \approx h \cdot \underbrace{\sum_{i=1}^n f(x_i)}_{\text{sum of right heights}}$$

- *mid-points approximation:*

$$\int_a^b f(x) dx \approx h \cdot \underbrace{\sum_{i=1}^n \frac{f(x_{i-1}) + f(x_i)}{2}}_{\text{sum of mid-point heights}}$$

Take the mid-points quadrature as an example. The algorithm goes as:

Algorithm 3 Quadrature via mid-points approximation

Require: Function $f(x)$, interval $[a, b]$, number of steps N

Ensure: Approximate integral $\int_a^b f(x) dx$

```

1:  $h \leftarrow (b - a) / N$  ▷ Step size
2:  $sum \leftarrow 0$ 
3: for  $i \leftarrow 0$  to  $N - 1$  do
4:    $x_i \leftarrow a + (i + 0.5) \cdot h$  ▷ Midpoint of sub-intervals
5:    $sum \leftarrow sum + f(x_i)$ 
6: end for
7:  $integral \leftarrow h \cdot sum$ 
8: return  $integral$ 

```

Coding Problem 1.11. (a) Write your code of quadrature via mid-point approximation. Make it a function that follows the Algorithm 3 above.

(b) Find some built-in quadrature functions in python and compare them with your function with some testing input functions.

³there are good reasons not to use the uniform grid

2. PDE

Read https://en.wikipedia.org/wiki/Partial_differential_equation, at least till ‘Classification’.

2.1. Heat equation (isotropic diffusion)

Definition 9. For a function $f(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}$, the *Laplacian* of f is

$$\Delta f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2} = f_{xx} + f_{yy}.$$

Similarly, for $f(x, y, z) : \mathbb{R}^3 \rightarrow \mathbb{R}$,

$$\Delta f = f_{xx} + f_{yy} + f_{zz}.$$

More generally, for $f(x_1, x_2, \dots, x_n) : \mathbb{R}^n \rightarrow \mathbb{R}$,

$$\Delta f = f_{x_1 x_1} + \dots + f_{x_n x_n} = \sum_{i=1}^n f_{x_i x_i}.$$

Definition 10. The n dimensional heat equation refers to

$$u_t = \Delta u, \tag{2.1}$$

where

- $u(t, x_1, \dots, x_n)$,
- $u_t = \frac{\partial u}{\partial t}$ (temporal derivative),
- $\Delta u = \sum_{i=1}^n u_{x_i x_i}$ (spatial Laplacian).

Problem 2.1. With $G(t, x) = \frac{1}{\sqrt{4\pi t}} e^{-\frac{x^2}{4t}}$ and some ‘good’ function $f(x)$, verify that the convolution

$$(G(t, \cdot) * f)(x) = \int_{-\infty}^{\infty} G(t, x - \tau) f(\tau) d\tau$$

solves the 1-D heat equation.

Extra question: re-write $t = \frac{\sigma^2}{2}$, does $G(t, x)$ now look like something you’ve seen before?

SOLUTION: $u(t, x) = (G(t, \cdot) * f)(x) = \int_{-\infty}^{\infty} G(t, x - \tau) f(\tau) d\tau$ solves the 1-D heat equation $\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$.

We have $G(t, x) = \frac{1}{\sqrt{4\pi t}} e^{-\frac{x^2}{4t}}$. The convolution is $\int_{-\infty}^{\infty} G(t, x - \tau) f(\tau) d\tau$.

1. Calculate $\frac{\partial u}{\partial t}$:

$$\frac{\partial u}{\partial t} = \frac{\partial}{\partial t} \int_{-\infty}^{\infty} G(t, x - \tau) f(\tau) d\tau = \int_{-\infty}^{\infty} \frac{\partial}{\partial t} G(t, x - \tau) f(\tau) d\tau$$

By product rule, $\frac{\partial G}{\partial t} = \left(-\frac{1}{2t} + \frac{(x-\tau)^2}{4t^2}\right) G(t, x-\tau)$. So,

$$\frac{\partial u}{\partial t} = \int_{-\infty}^{\infty} \left(-\frac{1}{2t} + \frac{(x-\tau)^2}{4t^2}\right) G(t, x-\tau) f(\tau) d\tau$$

2. Calculate $\frac{\partial^2 u}{\partial x^2}$:

$$\frac{\partial u}{\partial x} = \frac{\partial}{\partial x} \int_{-\infty}^{\infty} G(t, x-\tau) f(\tau) d\tau = \int_{-\infty}^{\infty} \frac{\partial}{\partial x} G(t, x-\tau) f(\tau) d\tau$$

$\frac{\partial G}{\partial x} = -\frac{x-\tau}{2t} G(t, x-\tau)$. So,

$$\frac{\partial u}{\partial x} = \int_{-\infty}^{\infty} -\frac{x-\tau}{2t} G(t, x-\tau) f(\tau) d\tau$$

And the second derivative:

$$\frac{\partial^2 u}{\partial x^2} = \frac{\partial}{\partial x} \int_{-\infty}^{\infty} -\frac{x-\tau}{2t} G(t, x-\tau) f(\tau) d\tau = \int_{-\infty}^{\infty} \frac{\partial}{\partial x} \left(-\frac{x-\tau}{2t} G(t, x-\tau)\right) f(\tau) d\tau$$

Again by product rule, $\frac{\partial}{\partial x} \left(-\frac{x-\tau}{2t} G(t, x-\tau)\right) = \left(-\frac{1}{2t} + \frac{(x-\tau)^2}{4t^2}\right) G(t, x-\tau)$. So,

$$\frac{\partial^2 u}{\partial x^2} = \int_{-\infty}^{\infty} \left(-\frac{1}{2t} + \frac{(x-\tau)^2}{4t^2}\right) G(t, x-\tau) f(\tau) d\tau$$

3. Compare $\frac{\partial u}{\partial t}$ and $\frac{\partial^2 u}{\partial x^2}$:

Comparing the results for $\frac{\partial u}{\partial t}$ and $\frac{\partial^2 u}{\partial x^2}$, we see that:

$$\frac{\partial u}{\partial t} = \int_{-\infty}^{\infty} \left(-\frac{1}{2t} + \frac{(x-\tau)^2}{4t^2}\right) G(t, x-\tau) f(\tau) d\tau$$

$$\frac{\partial^2 u}{\partial x^2} = \int_{-\infty}^{\infty} \left(-\frac{1}{2t} + \frac{(x-\tau)^2}{4t^2}\right) G(t, x-\tau) f(\tau) d\tau$$

Therefore,

$$\frac{\partial u}{\partial t} = \frac{\partial^2 u}{\partial x^2}$$

Remark 2.1. $f(x)$ needs to be good enough to guarantee the convergence of the convolution (an improper integral). In practice, $f(x)$ is the initial condition of the heat equation, namely,

$$u(0, x) = f(x).$$

Coding Problem 2.2. Use your code from Problem 1.11 to implement the convolution between

$$f(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$

and $G(t, x)$ in Problem 2.1 above with

- (a) $t = 0.1$,
- (b) $t = 1$,
- (c) $t = 10$.

Compare the outcomes.

Since you need a finite interval $[a, b]$ to implement the algorithm, try a few choices. What's your conclusion?

Coding Problem 2.3. Similar to the problem above, we now implement the 2D convolution. Consider the function

$$f(x, y) = \begin{cases} 1, & (x, y) \in [-1, 1] \times [-1, 1] \equiv \{(x, y) \mid -1 \leq x \leq 1, -1 \leq y \leq 1\} \\ 0, & \text{elsewhere.} \end{cases}$$

and the [2D Gaussian/heat kernel](#)

$$G(t, x, y) = \frac{1}{4\pi t} e^{-\frac{x^2+y^2}{4t}}.$$

Again, try different values of t and compare the outcomes.

Coding Problem 2.4. Now we are ready for the first image processing example.

- (a) Import a picture and convert it into a grayscale version, which makes generates a matrix. Expand the matrix by 0 beyond the boundaries.

Consider

- `np.linspace`
- `np.meshgrid`
- `np.where`

- (b) Select a σ .

Consider

- `np.exp`
- `np.sum`
- `np.matmul` (or equivalently “`*`” for matrix multiplication)

- (c) Use a convolution with the Gaussian to smooth/blur the graph.
- (d) Display the resulting matrix as a graph. Consider

- `plt.imshow`
- `plt.colorbar`

Coding Problem 2.5. Do it for just a portion of the picture.

Coding Problem 2.6. Now do it for a colored picture using *RGB format*.

- (a) Import a picture and convert it into a RGB version, which generates a 3D matrix. The 3rd dimension is of length 3.

Consider

- `from PIL import Image`
- `Image.open(path).convert('RGB')`

- (b) Separate the matrix into 3 matrices. For each of them, do the same as in the greyscale version.

Consider `np.split`.

- (c) Combine the resulting 3 matrices back into 1 matrix (RGB), and display.

Consider `np.stack`.

Definition 11. An *operator* is an extension of the concept of functions. It maps from one space X (domain) to another space Y (codomain) satisfying that each element in the domain gets mapped to a unique element in the codomain. $f(X) \subset Y$ is called the image.

Read more here: [https://en.wikipedia.org/wiki/Operator_\(mathematics\)](https://en.wikipedia.org/wiki/Operator_(mathematics))

Examples of operators:

- Any real-valued function $f(x)$ is an operator from $\mathbb{R}$ to $\mathbb{R}$.
- Any real-valued function $f(x, y)$ is an operator from $\mathbb{R}^2$ to $\mathbb{R}$.
- The derivative $\frac{d}{dx}$ is an operator from the space of differentiable functions to continuous functions (an advanced result says the derivative cannot have jump discontinuity).
- The derivative $\frac{d}{dx}$ is an also operator from the space of polynomials to itself.
- The anti-derivative $\int \cdot dx$ is NOT an operator on polynomials.
- The anti-derivative $\int \cdot dx$ with respect to $f(0) = 0$ is an operator on polynomials.
- The Laplacian is an operator on twice-differentiable functions.

Definition 12. An operator f is *linear* (over the real number field $\mathbb{R}$) if $f(\alpha x + \beta y) = \alpha f(x) + \beta f(y)$.

An operator f is *affine* if it's a translation of a linear map. In another word, the map from $b - a$ to $f(b) - f(a)$ is a linear map.

Examples:

- All linear maps are affine; no vice versa. See the one next.
- $f(x) = mx + b$ with $b \neq 0$ is NOT linear (check the definition); it is affine.

- $f(x) = mx$ is linear (therefore also affine).
- $\frac{d}{dx}$ is a linear operator.
- Any matrix (with respect to multiplication with vectors) is a linear operator.

Definition 13. An operator $\mathcal{L}$ for function $f(x, y) : \mathbb{R}^2 \rightarrow \mathbb{R}$ is *isotropic* if it's invariant (remaining the same) under rotation. If R is a rotation matrix in $\mathbb{R}^2$, then for any $\mathbf{v} = \begin{bmatrix} x & y \end{bmatrix}^T \in \mathbb{R}^2$,

$$\mathcal{L}[f(\mathbf{v})] = \mathcal{L}[f(R\mathbf{v})].$$

Otherwise, the operator is *anisotropic*.

This can be extended to higher dimension: the operator $\mathcal{L}$ for functions $f : \mathbb{R}^n \rightarrow \mathbb{R}$ is isotropic if it is invariant under a 'rotation',⁴

Problem 2.7. It's well known that in $\mathbb{R}^2$, any rotation can be represented by the matrix

$$R = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix}$$

Verify that the 2D Laplacian operator $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ is isotropic.

SOLUTION:

To verify that the operator is isotropic we must prove that it remains true even after rotation.

$$\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$$

we must rotate this and prove the equation holds true x' and y' will be our rotated values

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) \\ \sin(\theta) & \cos(\theta) \end{bmatrix} \cdot \begin{bmatrix} x \\ y \end{bmatrix}$$

$$x' = x \cos \theta + y \sin \theta$$

$$y' = -x \sin \theta + y \cos \theta$$

These are our respective derivative formulas

$$\frac{\partial}{\partial x} = \frac{\partial}{\partial x'} \frac{\partial x'}{\partial x} + \frac{\partial}{\partial y'} \frac{\partial y'}{\partial x}$$

$$\frac{\partial}{\partial y} = \frac{\partial}{\partial x'} \frac{\partial x'}{\partial y} + \frac{\partial}{\partial y'} \frac{\partial y'}{\partial y}$$

now plugging in the values of $\frac{\partial x'}{\partial x}, \frac{\partial y'}{\partial x}, \frac{\partial x'}{\partial y}, \frac{\partial y'}{\partial y}$, to the Laplacian $\Delta = \frac{\partial^2}{\partial x^2} + \frac{\partial^2}{\partial y^2}$ we get

$$\Delta = \left(\frac{\partial}{\partial x'} \cos \theta - \frac{\partial}{\partial y'} \sin \theta \right)^2 + \left(\frac{\partial}{\partial x'} \sin \theta + \frac{\partial}{\partial y'} \cos \theta \right)^2$$

$$\Delta = \frac{\partial^2}{\partial x'^2} \cos^2 \theta - 2 \frac{\partial}{\partial x'} \frac{\partial}{\partial y'} \sin \theta \cos \theta + \frac{\partial^2}{\partial y'^2} \sin^2 \theta + \frac{\partial^2}{\partial x'^2} \sin^2 \theta + 2 \frac{\partial}{\partial x'} \frac{\partial}{\partial y'} \sin \theta \cos \theta + \frac{\partial^2}{\partial y'^2} \cos^2 \theta$$

⁴rotation in higher dimension needs to be carefully defined.

the two $2 \frac{\partial}{\partial x'} \frac{\partial}{\partial y'} \sin \theta \cos \theta$ cancel out since one is negative and one is positive and we can group and factor the $\frac{\partial^2}{\partial x'^2}$ and $\frac{\partial^2}{\partial y'^2}$ to give us

$$\Delta = \frac{\partial^2}{\partial x'^2} (\sin^2 \theta + \cos^2 \theta) + \frac{\partial^2}{\partial y'^2} (\sin^2 \theta + \cos^2 \theta)$$

$(\sin^2 \theta + \cos^2 \theta) = 1$ and that leaves us with

$$\Delta = \frac{\partial^2}{\partial x'^2} + \frac{\partial^2}{\partial y'^2}$$

just like our original equation but with the rotated values instead, proving that it is isotropic

Definition 14. for the 1 dimensional case of the heat equation with the heat equation we know $\frac{\partial}{\partial t} = \frac{\partial^2}{\partial x^2}$ we can use $\frac{u_{x,t+1} - u_{x,t}}{\Delta t}$ to approximate $\frac{\partial}{\partial t}$ and $\frac{(u_{x+1,t+1} - u_{x,t+1}) - (u_{x,t+1} - u_{x-1,t+1})}{\Delta x}$ to approximate $\frac{\partial^2}{\partial x^2}$

SOLUTION: We can plug in these approximations to the heat equation and re arrange to get

$$\frac{u_{x,t+1} - u_{x,t}}{\Delta t} = k^2 * \frac{(u_{x+1,t+1} - u_{x,t+1}) - (u_{x,t+1} - u_{x-1,t+1})}{\Delta x}$$

2.2. Anisotropic Diffusion

Definition 15. Let $u(x, y, t)$ be a grayscale image evolving over time $t \geq 0$, with $u(x, y, 0) = f(x, y)$ the original image. The *Perona–Malik diffusion equation* is:

$$\frac{\partial u}{\partial t} = \nabla \cdot (c(|\nabla u|) \nabla u), \quad (2.2)$$

where:

- $\nabla u = \left(\frac{\partial u}{\partial x}, \frac{\partial u}{\partial y} \right)$ is the image gradient,
- $|\nabla u|$ measures the strength of edges,
- $c(s)$ is the *diffusivity function*, which depends on the gradient magnitude.

The function $c(s)$ controls how diffusion behaves at different gradient magnitudes. Perona and Malik proposed⁵ two typical forms:

(a) exponential:

$$c(s) = \exp \left(- \left(\frac{s}{K} \right)^2 \right)$$

⁵P. Perona and J. Malik, *Scale-space and edge detection using anisotropic diffusion*, IEEE Transactions on Pattern Analysis and Machine Intelligence, 12(7), 1990.

(b) inverse quadratic:

$$c(s) = \frac{1}{1 + \left(\frac{s}{K}\right)^2}$$

Here, $K > 0$ is a contrast parameter that controls the sensitivity to edges:

- For small gradients $s \ll K$, $c(s) \approx 1$: diffusion is active (smoothing). Therefore, in ‘flat’ regions (low $|\nabla u|$), the diffusion behaves like the heat equation, smoothing noise.
- For large gradients $s \gg K$, $c(s) \approx 0$: diffusion is suppressed (edge-preserving). Therefore, near edges (high $|\nabla u|$), the diffusivity is low, preventing smoothing across edges.

In sum, this behavior leads to anisotropic diffusion: diffusion strength depends on the direction and location in the image.

Coding Problem 2.8. Find the discrete form of (2.2) and implement it on a greyscale picture, following [this link](#), slides 23-29; 33.

- Zero Neumann BC (no flux).
- Extra: Dirichlet BC (same as the initial image).

SOLUTION: Suppose the image is represented on a 2D grid $u_{i,j}^n$, where:

- (i, j) are pixel indices,
- n denotes the time (iteration) index,
- Δt is the time step.

The continuous PDE:

$$\frac{\partial u}{\partial t} = \nabla \cdot (c(|\nabla u|) \nabla u)$$

is approximated by a finite difference scheme.

Step 1: Discrete Gradients and Diffusivity.

We define the discrete gradients using forward differences in four directions (North, South, East, West):

$$\begin{aligned}\nabla_N u_{i,j} &= u_{i-1,j} - u_{i,j}, \\ \nabla_S u_{i,j} &= u_{i+1,j} - u_{i,j}, \\ \nabla_E u_{i,j} &= u_{i,j+1} - u_{i,j}, \\ \nabla_W u_{i,j} &= u_{i,j-1} - u_{i,j}.\end{aligned}$$

We compute the diffusivity for each direction using a chosen function $c(s)$:

$$c_N = c(|\nabla_N u|), \quad c_S = c(|\nabla_S u|), \quad c_E = c(|\nabla_E u|), \quad c_W = c(|\nabla_W u|).$$

From there, we get the explicit FD scheme for (2.2):

$$u_{i,j}^{(k+1)} = u_{i,j}^{(k)} + \Delta t [c_N \nabla_N u_{i,j} + c_S \nabla_S u_{i,j} + c_W \nabla_W u_{i,j} + c_E \nabla_E u_{i,j}] \quad (2.3)$$

Coding Problem 2.9. Test Perona-Malik on standard IEEE images (<https://www.kaggle.com/datasets/saeedehkamjoo/standard-test-images>).

3. Variational Methods

Next, we introduce the *Calculus of Variations*, a field of mathematical analysis that developed after and builds upon classical Calculus. Read more [here](#).

Definition 16. An *functional* is an operator from a space of functions to $\mathbb{R}$.

Examples:

- Given $a < b$, then for any continuous (or integrable in general) functions over the interval $[a, b]$,

$$I_1(f) = \int_a^b f(x) dx$$

- Given $a < b$, then for any continuous (or square integrable in general) functions over the interval $[a, b]$,

$$I_2(f) = \int_a^b f^2(x) dx.$$

- For any differentiable function $f(x)$ at $x = a$,

$$I_3(f) = f'(a).$$

- For any continuous functions over the interval $[a, b]$ and any $p \in [1, \infty)$,

$$N_p(f) = \left(\int_a^b |f(x)|^p dx \right)^{\frac{1}{p}} \quad (:= \|f\|_p).$$

- Given $a < b$ and a continuous function $g(x)$,

$$I_g(f) = \int_a^b f(x)g(x) dx \quad (:= \langle f, g \rangle)$$

defines a functional for any continuous (square integrable, in general) function $f(x)$.

Definition 17. Similar to Definition 12, we say a functional $F(\cdot)$ is *linear* if $F(\alpha f + \beta g) = \alpha F(f) + \beta F(g)$ for any $\alpha, \beta \in \mathbb{R}$.

Problem 3.1. Which examples after Definition 16 are linear?

Definition 18. Given a multivariate function $f(x)$ ($x \in \mathbb{R}^n$), its *total variation over a domain* $\Omega \subset \mathbb{R}^n$ is a functional defined by the following multivariate integral

$$TV(f) = \int_{\Omega} |\nabla f(x)| dx. \quad (3.1)$$

Remark 3.1. The total variation of a function measures how much the function $f(x)$ varies across the domain. In the context of images, we know from the Perona-Malik model (15) that it preserves edges while penalizing oscillations.

Definition 19. The *Variational principle* states

Nature evolves in a way that extremizes (often minimizes) some quantity.

Such quantity is a functional. In classical mechanics, the path taken by a particle is the one that minimizes (or makes stationary) a certain integral quantity. Read more [here](#).

Remark 3.2. The heat equation (classic, isotropic version) can be derived from the minimization of the Dirichlet energy $E(u) = \frac{1}{2} \int_{\Omega} |\nabla u|^2 dx$, which is a second order total variation. The reason for the second order comes from how entropy is defined from thermal energy and can be found in classic physics textbooks.

3.1. Euler-Lagrange equation

One of the ways to realize the variational principle is through the *Euler-Lagrange equation*. We simplify it as follows.

Definition 20. Given an integral functional⁶ $E(u)$ for some function $u(x)$, we consider a test function (as smooth as we need) $v(x)$, and define the *first variation of E with respect to v* as the *Gateaux derivative* of $E[u]$ with respect to v :

$$\delta E(u)[v] := \left. \frac{d}{d\epsilon} E[u + \epsilon v] \right|_{\epsilon=0}. \quad (3.2)$$

Since the functional is in an integral form (also called a *weak form*⁷), the computed derivative would always be in a form (guaranteed by physics and by Emmy Noether's theorems⁸)

$$\delta E(u)[v] = \int_{\Omega} F(u)v dx. \quad (3.3)$$

Key Fact: if minimization of E is obtained, then the derivative should be 0 with respect to any v . Hence, $F(u) = 0$. This is the main idea behind the Euler-Lagrange equation and is a generalization of “critical points” for weak form equations.

Whether a “critical point” is a maximum, a minimum, a saddle point, or something else depends on a “second derivative test”. In finite dimensional case, it's the *Hessian matrix*⁹.

Hence, we have the following definition:

Definition 21. The functional $F(u)$ in (3.3) above is the *total variation of $E(u)$ with respect to u* and is denoted as $\frac{\delta E}{\delta u}$. In particular,

- When minimization is obtained, $\frac{\delta E}{\delta u} = 0$.

⁶it means the functional must be defined as an integral of the function u . It enforces that the functional stands for some energy.

⁷intuitively, an integral equation $\int_{\Omega} u(x) dx = \int_{\Omega} v(x) dx$ is weaker than a pointwise equation $u(x) = v(x)$.

⁸Talk to Cecily about this,

⁹Talk to Delaney about this.

- We can obtain the minimum via the *gradient flow*:

$$\frac{\partial u}{\partial t} = -\frac{\delta E}{\delta u}. \quad (3.4)$$

Problem 3.2. *Given*

$$E(u) = \int_{\Omega} u^2 dx, \quad (3.5)$$

verify that the first variation of E is $\frac{\delta E}{\delta u} = 2u$.

Problem 3.3. *Given*

$$E(u) = \int_{\Omega} |\nabla u|^2 dx, \quad (3.6)$$

verify that the first variation of E is $\frac{\delta E}{\delta u} = -2\Delta u$.

You will need the first Green's identities (a multivariate Integration-by-parts, to be covered in MATH 263):

$$\int_{\Omega} \nabla u \cdot \nabla v dx = \int_{\partial\Omega} v \nabla u \cdot \mathbf{n} d\partial\Omega - \int_{\Omega} v \Delta u dx$$

We can also assume Neumann boundary conditions for u so that $\int_{\partial\Omega} v \nabla u \cdot \mathbf{n} d\partial\Omega = 0$.

Problem 3.4. *Let $u(x) : \mathbb{R}^2 \rightarrow \mathbb{R}$. Given*

$$E(u) = \int_{\Omega} |\nabla u| dx, \quad (3.7)$$

verify that the first variation of E with respect to v is

$$\delta E(u)[v] = \int_{\Omega} \frac{\nabla u \cdot \nabla v}{|\nabla u|} dx,$$

and

$$\frac{\delta E}{\delta u} = -\nabla \cdot \left(\frac{\nabla u}{|\nabla u|} \right).$$

Again, We can also assume Neumann boundary conditions for u so that $\int_{\partial\Omega} v \nabla u \cdot \mathbf{n} d\partial\Omega = 0$.

Hint: $|\nabla u| = (u_x^2 + u_y^2)^{\frac{1}{2}}$.

Problem 3.5. *Given a function f and*

$$E(u) = \frac{\lambda}{2} \int_{\Omega} \int_{\Omega} [u(x) - f(x)]^2 dx, \quad (3.8)$$

verify that $\frac{\delta E}{\delta u} = \lambda(u - f)$.

3.2. Application 1: Denoising (Restoration)

Rudin, Leonid I., Stanley Osher, and Emad Fatemi. "Nonlinear total variation based noise removal algorithms." *Physica D: nonlinear phenomena* 60, no. 1-4 (1992): 259-268. [PDF file](#)¹⁰

¹⁰The famous first ever image of a black hole in 2019 used this algorithm! See [here](#). -Cecily

3.2.1 Application 2: Deblurring

- Blind Deblurring

Chan, Tony F., and Chiu-Kwong Wong. “Total variation blind deconvolution.” *IEEE transactions on Image Processing* 7, no. 3 (1998): 370-375. [PDF Here](#)

- Non-blind Deblurring

Vogel, Curtis R., and Mary E. Oman. “Iterative methods for total variation denoising.” *SIAM Journal on Scientific Computing* 17, no. 1 (1996): 227-238. [PDF Here](#)

3.2.2 Application 3: Segmentation

Chan, Tony F., and Luminita A. Vese. “Active contours without edges.” *IEEE Transactions on image processing* 10, no. 2 (2001): 266-277. [PDF file](#)

3.3. Choice of function for Non linear Diffusion

Given information

1. $s \geq 0$
2. $b(s) = c(s) + 2sc'(s)$

Requirements

- (a) $c(s)$ is always positive
- (b) $c(s)$ is always decreasing
- (c) $\lim_{s \rightarrow \infty} c(s) \rightarrow 0$
- (d) $c(0) = 1$
- (e) $b(s) \geq 0$

I propose the general form of the equation that meets all the requirements.

$$c(s) = \frac{1}{1 + s^k} \quad (3.9)$$

where k is some constant $0 < k \leq 0.5$

We can see that so long as $k > 0$ requirements a through d are met for any choice of k , requirement e takes a bit more work

$$c'(s) = \frac{ks(k-1)}{-(1+s^k)^2} \quad (3.10)$$

$$b(s) = \frac{(1-2k)s^k + 1}{(1+s^k)^2} \quad (3.11)$$

As long as $1 - 2k \geq 0$, $b(s)$ is always positive, so we know $0.5 \geq k$

Limit of $\frac{b(s)}{c(s)}$ $c(s)$ and $b(s)$ control the amount of blurring in the tangential and normal directions, respectively. We know that as $\lim_{s \rightarrow \infty} c(s) \rightarrow 0$ and in order for the rest of the requirements to be met $b(s)$ must go to 0 as well.

They can go at different rates though, and this will affect the way in which our image is blurred given when there are high gradients.

The book tells us that $\lim_{s \rightarrow \infty} \frac{b(s)}{c(s)} \rightarrow 0$ but this does not seem to be necessary. For our formula the general form looks like this

$$\frac{b(s)}{c(s)} = \frac{(1 - 2k)s^k + 1}{1 + s^k} \quad (3.12)$$

$$\lim_{s \rightarrow \infty} \frac{b(s)}{c(s)} = \frac{(1 - 2k)}{1} \quad (3.13)$$

Our choice of k will affect the limit. $k = 0.5$ will give us zero like the book says, but we can also end up with other results